SINAI UNIVERSITY
KANTARA

INT 453: Digital Multimedia

2024-2025

Lecturer : Dr. Maysa ElNoby, Dr. Reham Amin
Reham.ali@su.edu.eg

Lecture 8

JPEG Compression

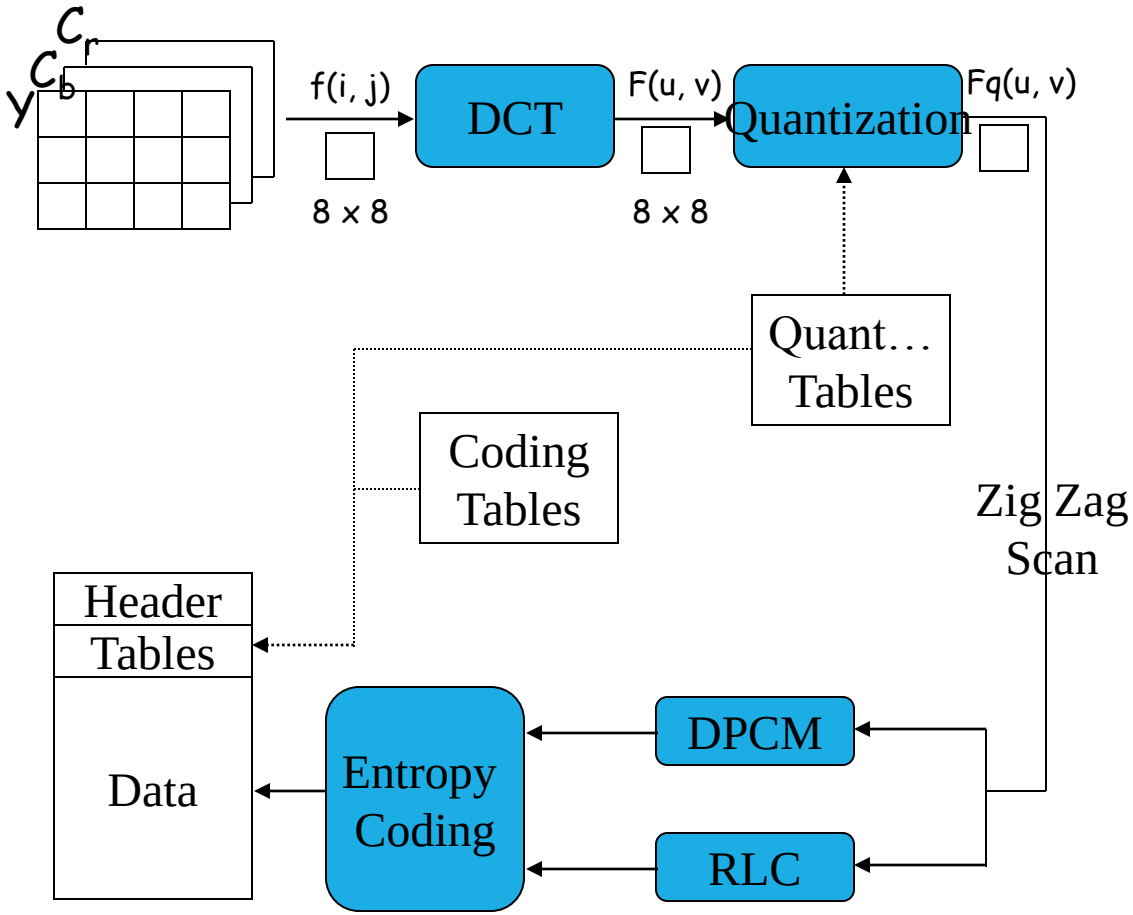
Why JPEG

The *compression ratio* of lossless methods (e.g., Huffman, Arithmetic, LZW) is not high enough for image and video compression.

JPEG uses *transform coding*, it is largely based on the following observations:

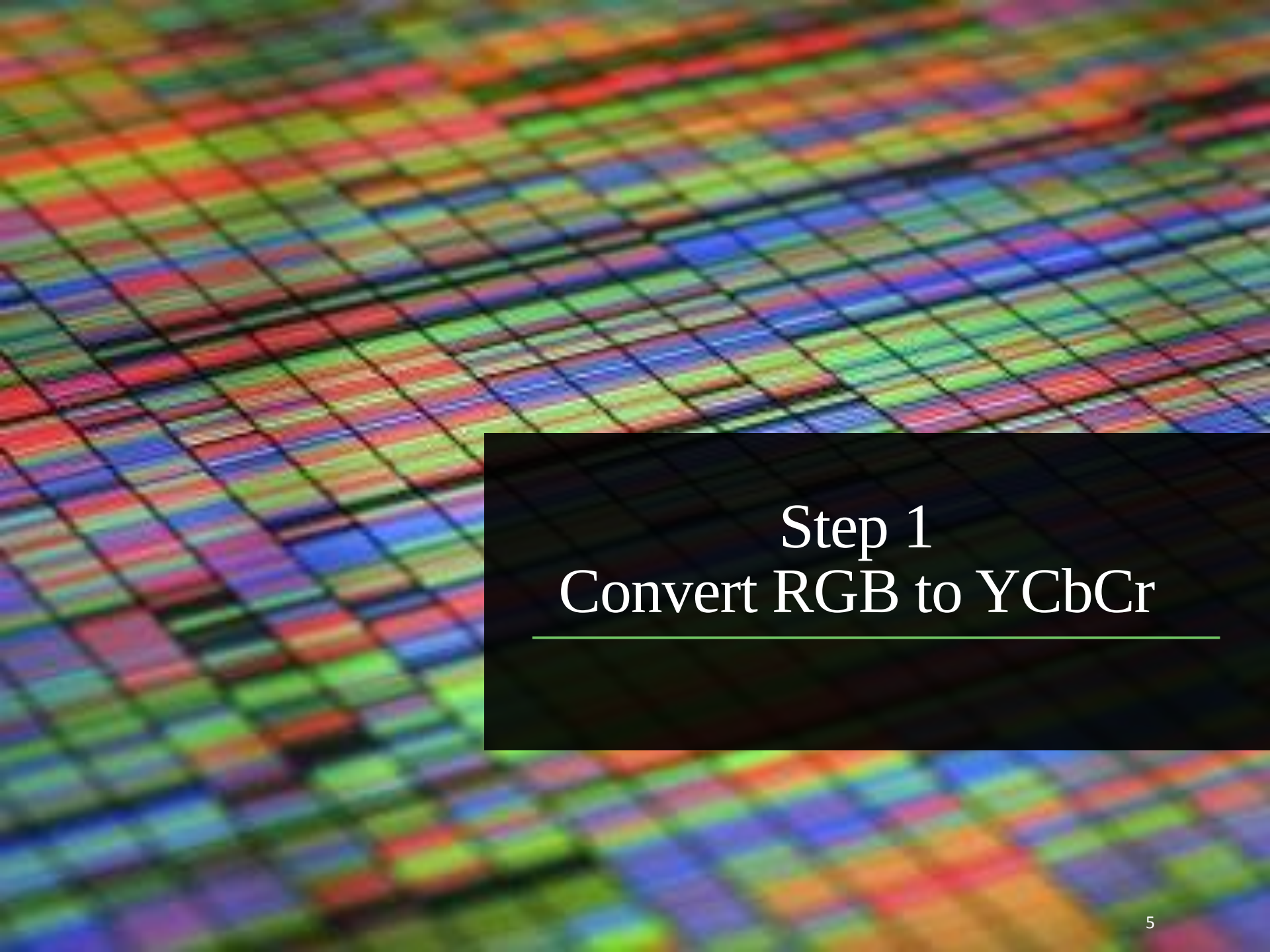
- Observation 1: A large majority of useful image contents change relatively slowly across images,
  - i.e., it is unusual for intensity values to alter up and down several times in a small area, for example, within an 8 x 8 image block.
  - For frequency domain, lower frequency components contain **more information** than the high frequency components which often correspond to less useful details and noises.
- Observation 2: Experiments suggest that humans are more immune to loss of higher frequency components than loss of lower frequency components.

JPEG Coding



Steps Involved:

1. Convert RGB to YCbCr
2. Discrete Cosine Transform of each 8x8 pixel array f(x,y) $\rightarrow_T$ F(u,v)
3. Quantization using a table or using a constant
4. Zig-Zag scan to exploit redundancy
5. Differential Pulse Code Modulation *(DPCM) on the DC component* and *Run length Coding of the AC* components
6. Entropy coding (Huffman) of the final output



Step 1
Convert RGB to YCbCr

1- RGB to YCrCb Color Conversion

RGB Color Model

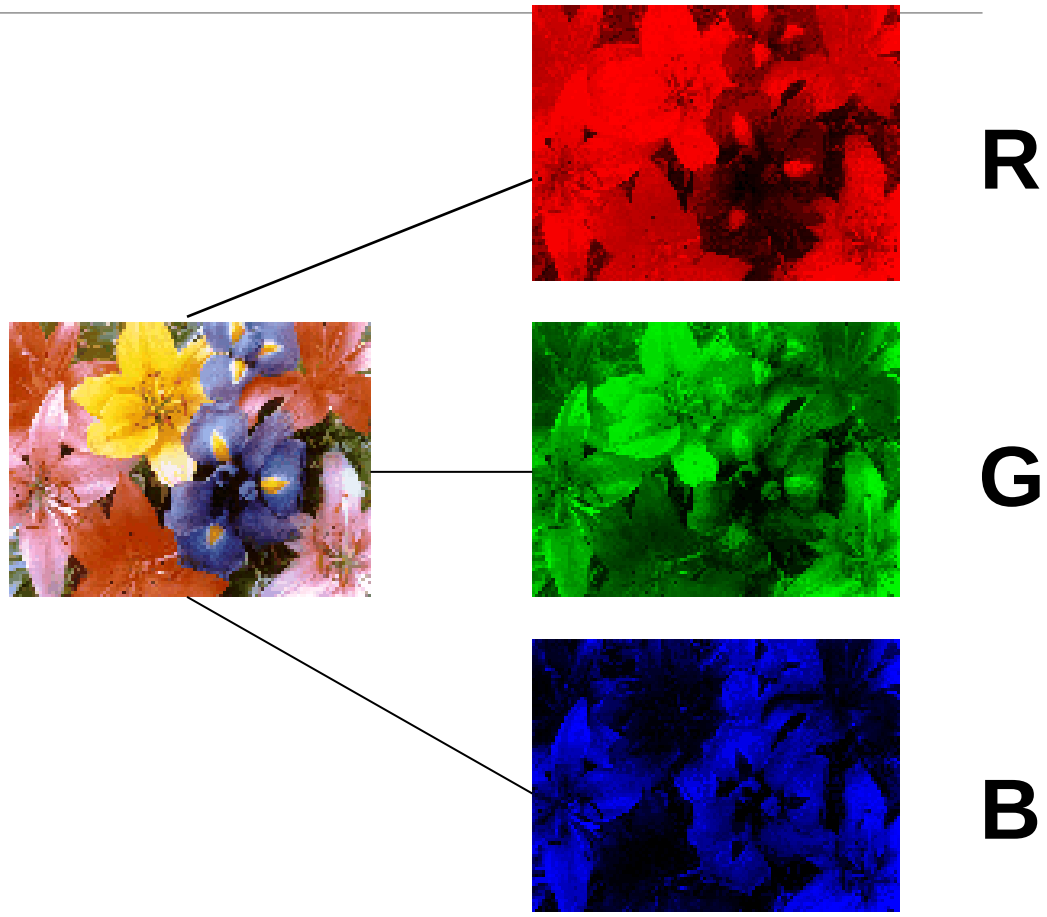
Three basic colors

R: Red

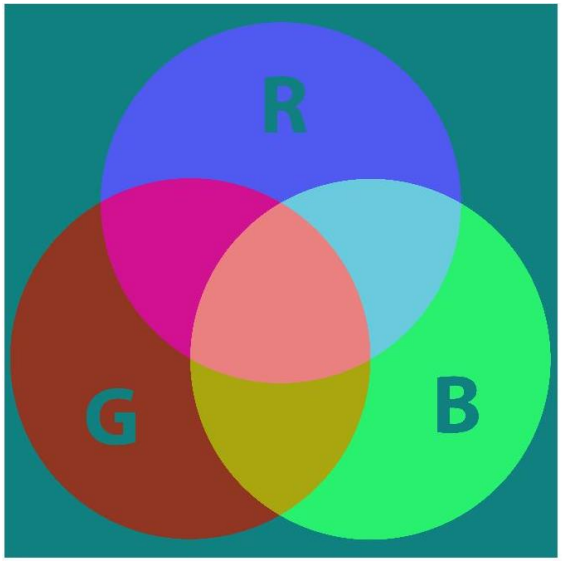
G: Green

B: Blue

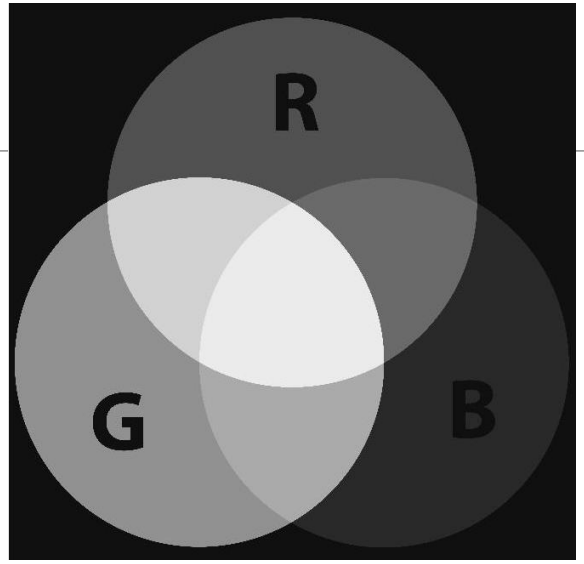
➔A picture consists of three images



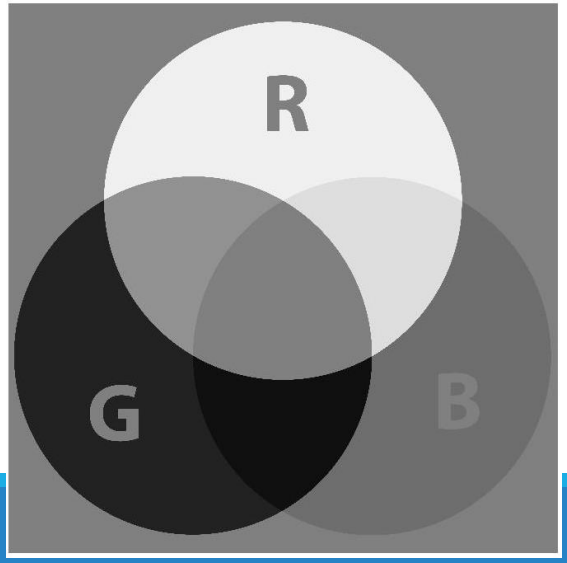
1- RGB to YCrCb Color Conversion (cont.,)



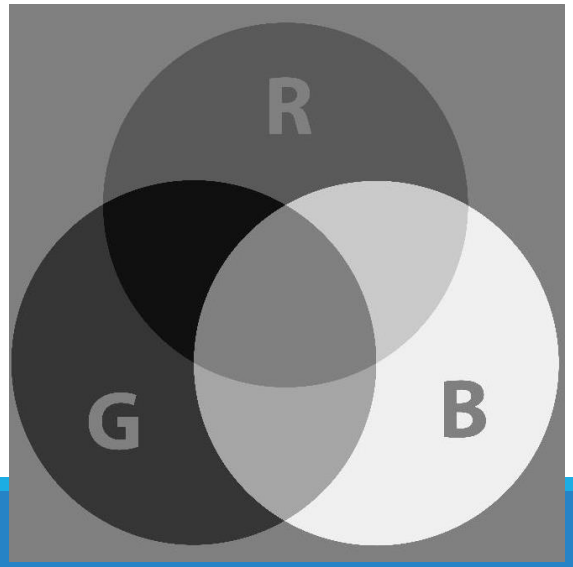
RGB



Y



Cr



Cb

YCrCb Color Model

YCrCb color model: used in JPEG and MPEG

Closely related to YUV: scaled and shifted YUV

Y - Luminance containing brightness and detail (monochrome TV)

To create the Y signal, the red, green and blue inputs to the Y signal must be balanced to compensate for the color perception misbalance of the eye.

- **Y = 0.3R + 0.59G + 0.11B**
- **Cr = ((R – Y)/1.6) + 0.5**
- **Cb = ((B – Y)/2) + 0.5**

Human eyes are most sensitive to Y

Chrominance value in YCrCb are always in the range of **0 to 1** (normalization)

→ *Make digital processing easy*

YCrCb Color Model

Image in RGB components is converted into Luminance-Chrominance components

Chrominance Components are Converting from RGB to YCb Cr

$$\begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} = \begin{pmatrix} 0.299000 & 0.587000 & 0.114000 \\ -0.168736 & -0.331264 & 0.500002 \\ 0.500000 & -0.418688 & -0.081312 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}.$$

CB & CR can result in negative. To bring it to positive & representable in 8 bits

$$\begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} = \begin{pmatrix} 0.29900 & 0.58700 & 0.11400 \\ -0.16874 & -0.33126 & 0.50000 \\ 0.50000 & -0.41869 & -0.08131 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} + \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix}$$

The inverse color transformation is

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1.0 & 0.0 & 1.40210 \\ 1.0 & -0.34414 & -0.71414 \\ 1.0 & 1.77180 & 0.0 \end{pmatrix} \begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} - \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix}$$

Step 2
Discrete Cosine Transform of each 8x8 pixel array

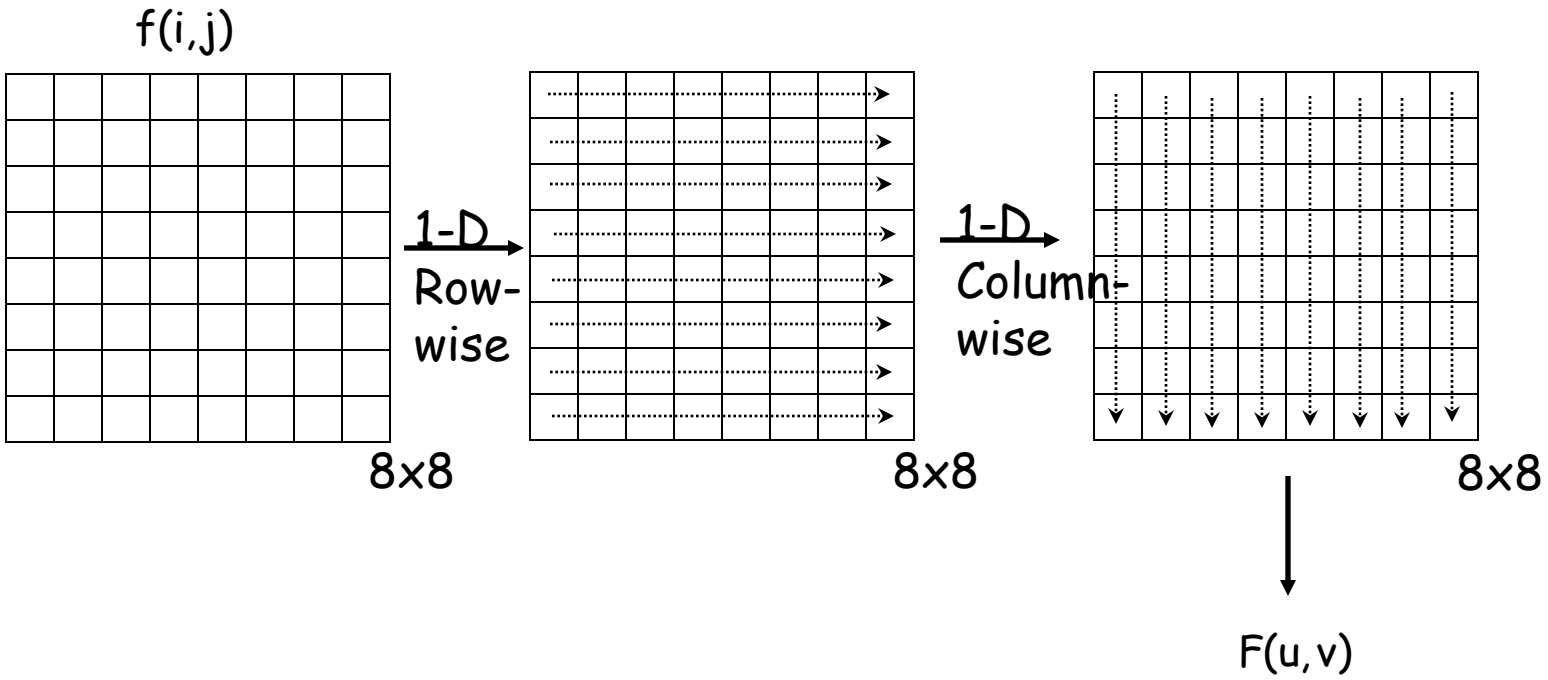
DCT on image blocks

- Each image is divided into 8x8 blocks.
- The 2D DCT is applied to each block image $f(i, j)$, yielding an output of the DCT coefficients $F(u, v)$ for each block.
- Using blocks, however, has the effect of isolating each block from its neighboring context.
- This is why JPEG images look choppy ("blocky") when a high *compression ratio* is specified by the user.

2-D DCT

Images are two-dimensional; How do you perform 2-D DCT?

- Two series of 1-D transforms result in a 2-D transform as demonstrated in the figure below



- F(0,0) is called *the DC component* and the rest of F(i,j) are called *AC components*

Equations for 2D DCT

Forward DCT:

$$F(u,v) = \frac{2}{\sqrt{nm}} C(u)C(v) \sum_{y=0}^{m-1} \sum_{x=0}^{n-1} I(x,y) * \cos\left(\frac{(2x+1)u\pi}{2n}\right) * \cos\left(\frac{(2y+1)v\pi}{2m}\right)$$

Inverse DCT:

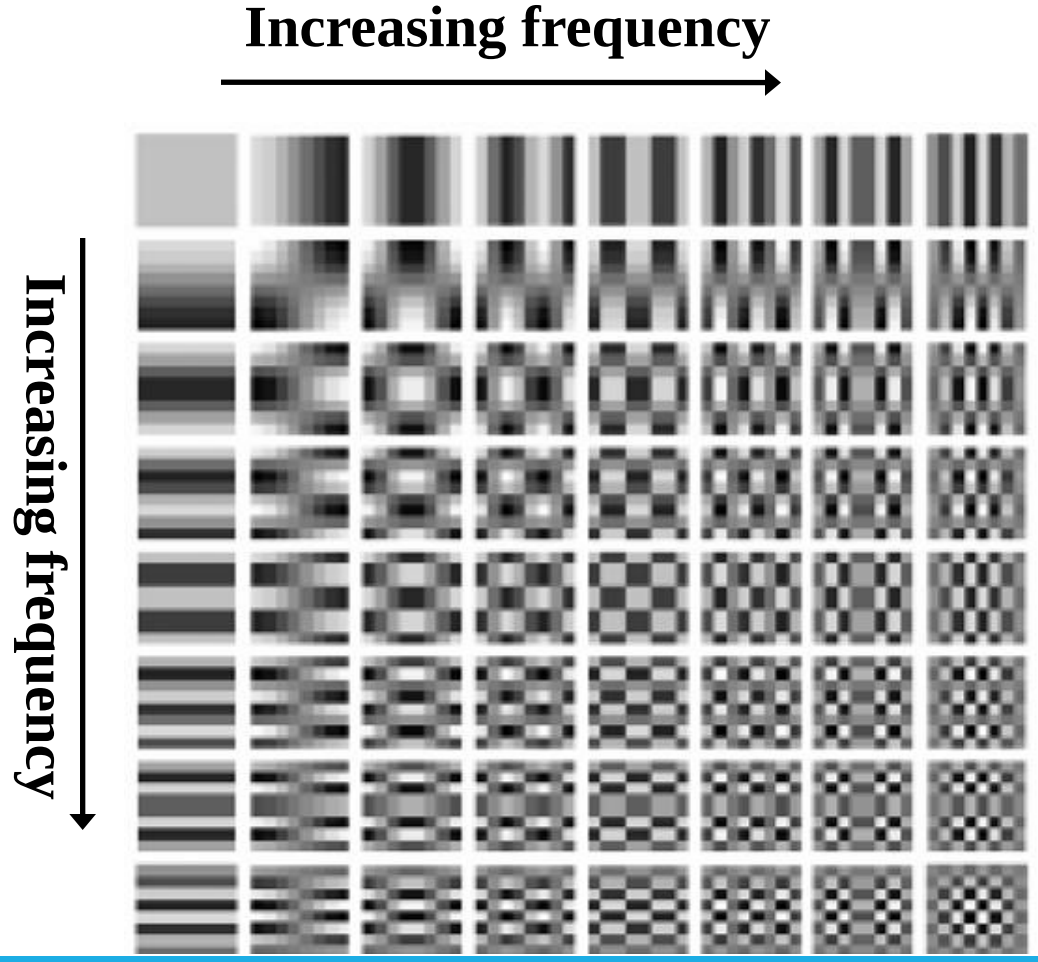
$$I(y,x) = \frac{2}{\sqrt{nm}} \sum_{v=0}^{m-1} \sum_{u=0}^{n-1} F(v,u)C(u)C(v) \cos\left(\frac{(2x+1)u\pi}{2n}\right) * \cos\left(\frac{(2y+1)v\pi}{2m}\right)$$

Optimized method to find 2D DCT

$$F(u, v) = t(u, i)\ \mathbf{f(x, y)}\ t^{T}(u, i)$$

$$t(u,i) = \begin{cases} \frac{1}{\sqrt{n}} & u = 0 \\ \sqrt{\frac{2}{n}} \cos\left(\frac{(2i+1)u\pi}{2n}\right) & u \neq 0 \end{cases}$$

Visualization of 2D DCT Basis Functions



Coefficient Differentiation

F(0,0)

- includes the lowest frequency in both directions
- The most useful information
- It can be observed by the human eye
- It is called **DC coefficient**
- Determines fundamental color of the block

F(0,1) …. F(7,7)

- are called **AC coefficients**
- Their frequency is non-zero in one or both directions

Step 3
Quantization using a table or using a constant

Quantization

$$\hat{F}(u,v) = round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

- $F(u, v)$ represents a DCT coefficient,
- $Q(u, v)$ is a quantization matrix, and
- $F'(u,v)$ represents the *quantized DCT coefficients* which JPEG will use in the succeeding entropy coding.
- **The quantization step is the *main source for loss* in JPEG compression**.
  - The entries of $Q(u, v)$ tend to have larger values towards the lower right corner.
  - This aims to introduce more loss at the higher spatial frequencies (a practice supported by Observations 1 and 2).

Quantization (Cont'd)

The next tables show the default $Q(u, v)$ values obtained from psychophysical studies with the goal of maximizing the compression ratio while minimizing perceptual losses in JPEG images.

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Luminance Quantization Table

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

Chrominance Quantization Table

Example: JPEG compression for a smooth image block



An 8×8 block from the Y image of 'Lena'

200	202	189	188	189	175	175	175
200	203	198	188	189	182	178	175
203	200	200	195	200	187	185	175
200	200	200	200	197	187	187	187
200	205	200	200	195	188	187	175
200	200	200	200	200	190	187	175
205	200	199	200	191	187	187	175
210	200	200	200	188	185	187	186

$f(i, j)$

515	65	-12	4	1	2	-8	5
-16	3	2	0	0	-11	-2	3
-12	6	11	-1	3	0	1	-2
-8	3	-4	2	-2	-3	-5	-2
0	-2	7	-5	4	0	-1	-4
0	-3	-1	0	4	1	-1	0
3	-2	-3	3	3	-1	-1	3
-2	5	-2	4	-2	2	-3	0

$F(u, v)$

Example: JPEG compression for a smooth image block (Cont'd)

32	6	-1	0	0	0	0	0
-1	0	0	0	0	0	0	0
-1	0	1	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\hat{F}(u,v)$

512	66	-10	0	0	0	0	0
-12	0	0	0	0	0	0	0
-14	0	16	0	0	0	0	0
-14	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\tilde{F}(u,v)$

199	196	191	186	182	178	177	176
201	199	196	192	188	183	180	178
203	203	202	200	195	189	183	180
202	203	204	203	198	191	183	179
200	201	202	201	196	189	182	177
200	200	199	197	192	186	181	177
204	202	199	195	190	186	183	181
207	204	200	194	190	187	185	184

$\tilde{f}(i,j)$

1	6	-2	2	7	-3	-2	-1
-1	4	2	-4	1	-1	-2	-3
0	-3	-2	-5	5	-2	2	-5
-2	-3	-4	-3	-1	-4	4	8
0	4	-2	-1	-1	-1	5	-2
0	0	1	3	8	4	6	-2
1	-2	0	5	1	1	4	-6
3	-4	0	6	-2	-2	2	2

$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

JPEG compression for a textured image block



Another 8×8 block from the Y image of 'Lena'

70	70	100	70	87	87	150	187
85	100	96	79	87	154	87	113
100	85	116	79	70	87	86	196
136	69	87	200	79	71	117	96
161	70	87	200	103	71	96	113
161	123	147	133	113	113	85	161
146	147	175	100	103	103	163	187
156	146	189	70	113	161	163	197

$f(i,j)$

-80	-40	89	-73	44	32	53	-3
-135	-59	-26	6	14	-3	-13	-28
47	-76	66	-3	-108	-78	33	59
-2	10	-18	0	33	11	-21	1
-1	-9	-22	8	32	65	-36	-1
5	-20	28	-46	3	24	-30	24
6	-20	37	-28	12	-35	33	17
-5	-23	33	-30	17	-5	-4	20

$F(u,v)$

JPEG compression for a textured image block (Cont'd)

-5	-4	9	-5	2	1	1	0
-11	-5	-2	0	1	0	0	-1
3	-6	4	0	-3	-1	0	1
0	1	-1	0	1	0	0	0
0	0	-1	0	0	1	0	0
0	-1	1	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\hat{F}(u,v)$

-80	-44	90	-80	48	40	51	0
-132	-60	-28	0	26	0	0	-55
42	-78	64	0	-120	-57	0	56
0	17	-22	0	51	0	0	0
0	0	-37	0	0	109	0	0
0	-35	55	-64	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\tilde{F}(u,v)$

70	60	106	94	62	103	146	176
85	101	85	75	102	127	93	144
98	99	92	102	74	98	89	167
132	53	111	180	55	70	106	145
173	57	114	207	111	89	84	90
164	123	131	135	133	92	85	162
141	159	169	73	106	101	149	224
150	141	195	79	107	147	210	153

$\tilde{f}(i,j)$

0	10	-6	-24	25	-16	4	11
0	-1	11	4	-15	27	-6	-31
2	-14	24	-23	-4	-11	-3	29
4	16	-24	20	24	1	11	-49
-12	13	-27	-7	-8	-18	12	23
-3	0	16	-2	-20	21	0	-1
5	-12	6	27	-3	2	14	-37
6	5	-6	-9	6	14	-47	44

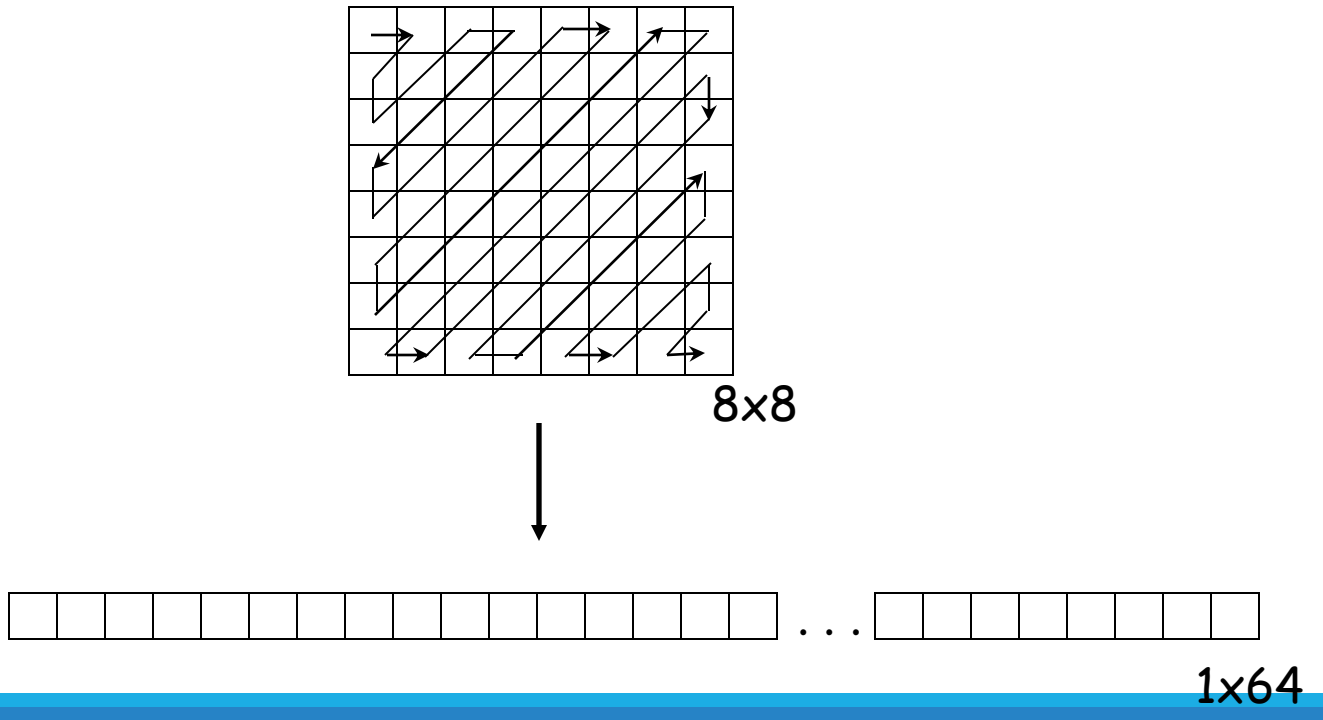
$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

Step 4
Zig-Zag scan to exploit redundancy

Zig-Zag Scan

Why? -- to group low frequency coefficients in top of vector and high frequency coefficients at the bottom

Maps 8 x 8 matrix to a 1 x 64 vector



Step 5

Apply *Run length Coding of the AC* components

Run-length Coding (RLC) on AC coefficients

RLC aims to turn the $F'(u,v)$ values into sets {number of *zeros, next non-zero values*}.

To make it most likely to hit a long run of zeros: a *zig-zag scan* is used to turn the 8x8 matrix $F'(u, v)$ into a *64-vector (arrow).*

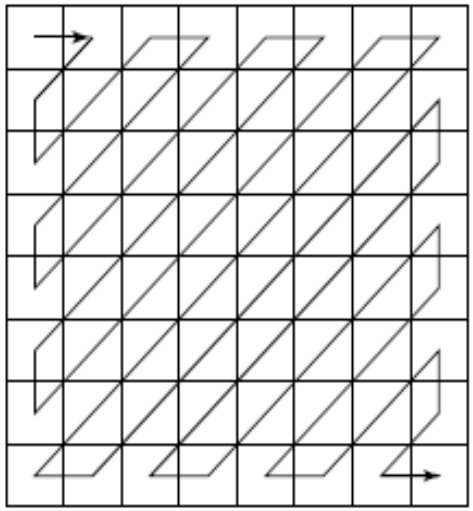


Fig. 9.4: Zig-Zag Scan in JPEG.

Run-length Coding (RLC) (Example)

- *Let F'(u, v) = (32, 6, -1, -1, 0, -1, 0, 0, 0, -1, 0, 0, 1, 0, 0, ……, 0)* with three runs o zeros in the middle and a run of 51 zeros at the end.
  - The RLC step replaces values by a pair (RUNLENGTH, VALUE) for each run o zeros in the AC coefficients of ***p,*** where:
    - RUNLENGTH is the number of zeros in the run
    - VALUE is the next nonzero coefficient.
  - To further save bits, a special pair (0, 0) indicates the end-of-block after the la nonzero AC coefficient is reached.

Run-length Coding (RLC) (Example)

- *Let F'(u, v) = (32, 6, -1, -1, 0, -1, 0, 0, 0, -1, 0, 0, 1, 0, 0, ……, 0)* with three runs o

  zeros in the middle and a run of 51 zeros at the end.

  - The RLC step replaces values by a pair (RUNLENGTH, VALUE) for each run o

    zeros in the AC coefficients of ***p,*** where:

    - RUNLENGTH is the number of zeros in the run
    - VALUE is the next nonzero coefficient.

  - In the above example, not considering the first (DC) component, we will thus have:

    - (0, 6)(0, -1)(0, -1)(1, -1)(3, -1)(2, 1)(0, 0)

Step 6
Differential Pulse Code Modulation *(DPCM) on the DC component*

DPCM on DC coefficients

The DC coefficients are coded separately from the AC ones.

Differential Pulse Code Modulation (DPCM) is the coding method.

Example:

- DC coefficients for the first 5 image blocks are 150, 155, 149, 152, 144
- In DPCM,

$$d_i = DC_{i+1} - DC_i \qquad d_0 = DC_0.$$

- d_0=150
- d_1=155 – 150 = 5
- d_2=149 – 155 = -6
- d_3=152 – 149 = 3
- d_4=144 – 152 = -8

Output = {150, 5, -6, 3, -8}

Step 7
Entropy coding (Huffman) of the final output

Entropy Coding

The DC and AC coefficients finally go through an entropy coding step to gain **a possible further compression.**

Huffman coding for DC coefficient is represented by

(SIZE, AMPLITUDE),

Where:

- SIZE indicates how many bits are needed for representing the coefficient, and
- AMPLITUDE contains the actual bits.

- SIZE is Huffman coded since smaller SIZEs occur much more often.
- AMPLITUDE is not Huffman coded, its value can change widely so Huffman coding has no appreciable benefit.

Entropy Coding

- Find the DPCM codes for the following DC coefficients:

150, 5, −6, 3, −8

- Huffman coding:

150 ----------→ (8, 10010110),

5 ----------→(3, 101),

-6 ----------→(3, 001),

3 ----------→(2, 11),

-8 ----------→(4, 0111)

Entropy Coding (Cont'd)

Baseline entropy coding details - size category.

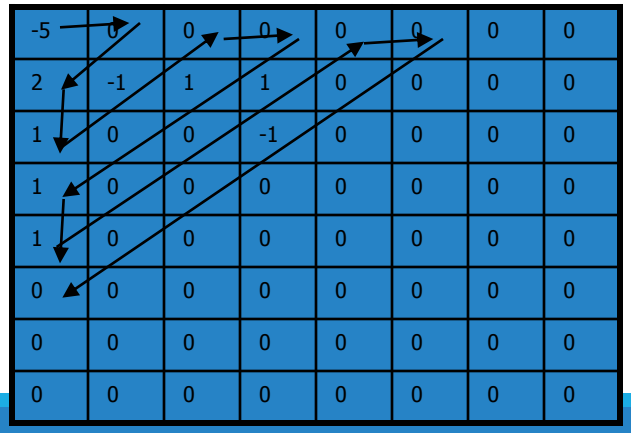
SIZE	AMPLITUDE
1	-1, 1
2	-3, -2, 2, 3
3	-7..-4, 4..7
4	-15..-8, 8..15
.	.
.	.
.	.
10	-1023..-512, 512..1023

Scanning and Huffman Encoding

Spatial Frequencies scanned in zig-zag pattern

(note high frequencies mostly zero)

Huffman encoding used to losslessly record values in table



0,2,1,-1,0,0,1,0,1,1,0,0,1,0,0,0,-1,0,0,… 0

All the AC cofficients in the zig zag ordering except the DC coefficient

Can be stored as:

(1,2),(0,1),(0,-1),(2,1),(1,1),(0,1)
,(2,1),(3,-1),EOB

Examples of varying JPEG compression ratios



500KB image, minimum compression



40KB image, half compression



11KB image, max compression

Close-up details of different JPEG compression ratios



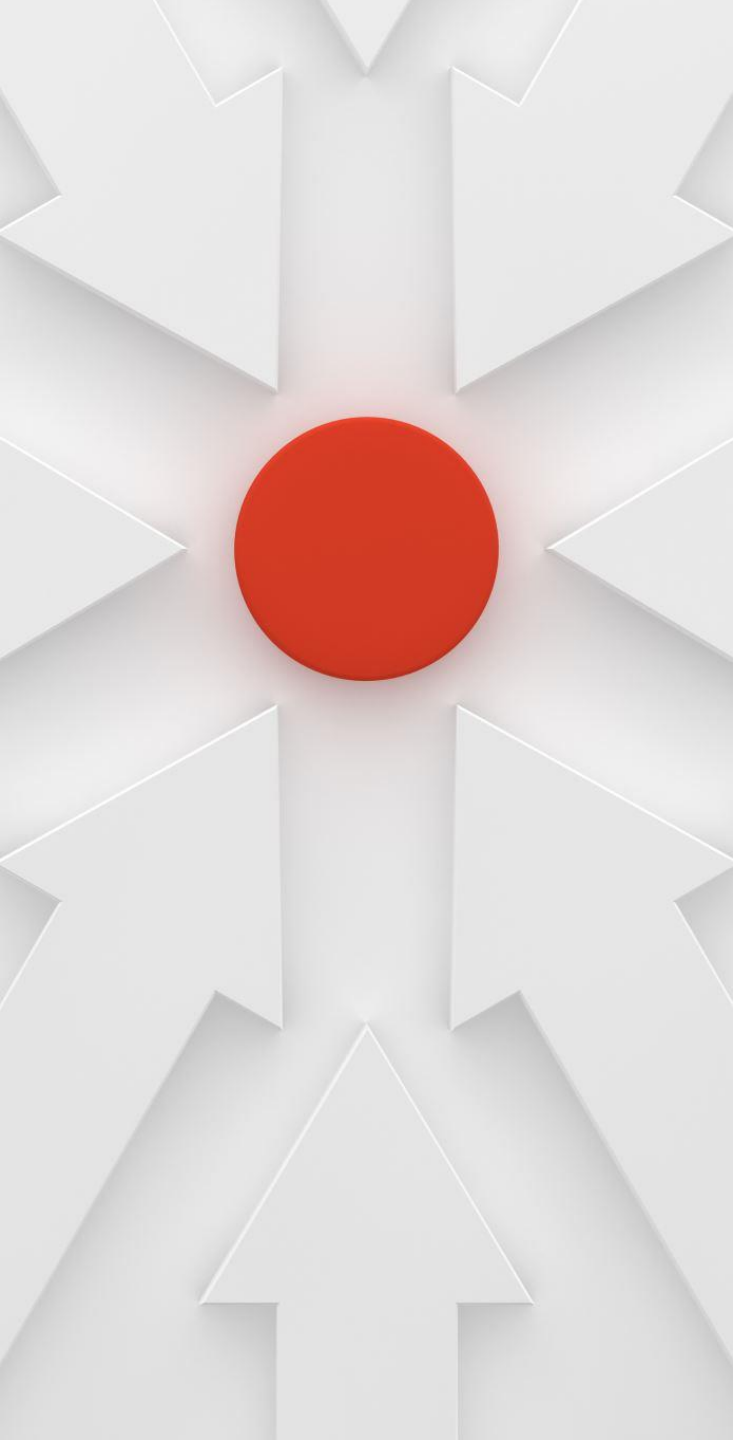
Uncompressed image (roughness between pixels still visible)



Half compression, blurring & halos around sharp edges



Max compression, 8-pixel blocks apparent, large distortion in high-frequency areas



JPEG Modes

- **Sequential Mode**
- **Lossless mode**
- **Progressive Mode**
- **Hierarchical Mode**

JPEG Modes: **Sequential Mode**

- Each image is encoded in a single left-to-right, top-to-bottom scan.
- The technique we have been discussing so far is an example of such a mode, also referred to as the **Baseline Sequential Mode.**
- It supports only 8-bit images as opposed to 12-bit images as described before.

Example - One Everyday Photo



2.76M

Example - One Everyday Photo



600K

Example - One Everyday Photo



350K

Example - One Everyday Photo



240K

Example - One Everyday Photo



144K

Example - One Everyday Photo



88K

The JPEG2000 standard

- It is also based on the wavelet transform
- It provides high quality images at very high compression ratios.
- The JPEG2000 compression method begins by level shifting the data to center it at zero, followed by an optional transform to decorrelate the data, such as a color transform for color images.
- The one-dimensional wavelet transform is applied to the rows and columns, and the coefficients are quantized based on the image size and number of wavelet bands utilized
- These quantized coefficients are then arithmetically coded on a bitplane basis
- It is designed to:
  - provide better compression than the DCT-based JPEG algorithm
  - Allow for progressive transmission of high quality images
  - Be able to compress binary and continuous tone images by allowing 1 to 16 bits for image components
  - allow random access to subimages

The JPEG2000 standard (cont.,)



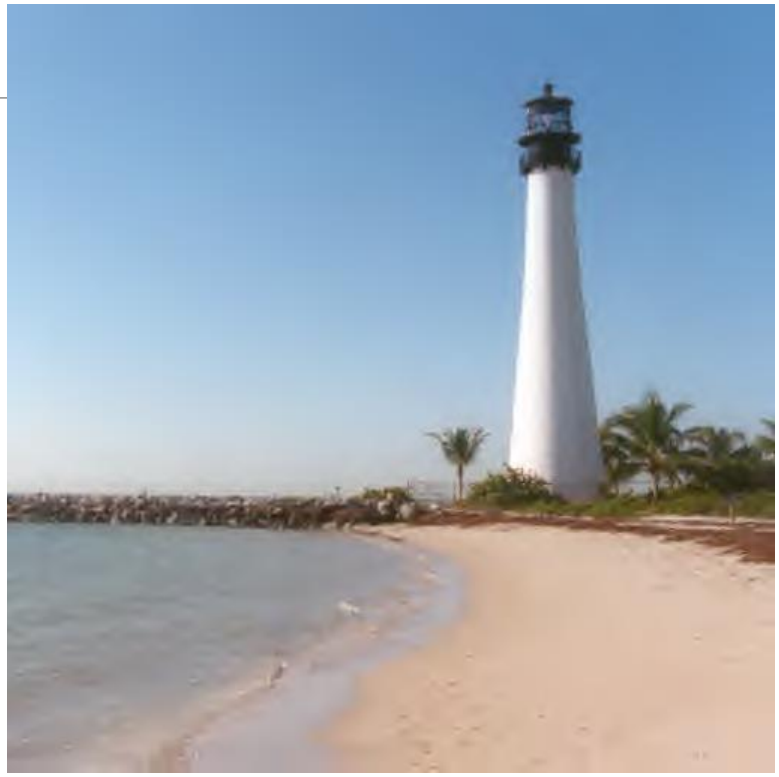
a) The original image

The JPEG2000 Algorithm Applied to a Color Image

The JPEG2000 standard (cont.,)



b) Compression ratio = 130 ,
compare to Fig10.3-21e (next slide)



c) Compression ratio = 200,
compare to Fig10.3-21f

The JPEG2000 Algorithm Applied to a Color Image

The JPEG2000 standard (cont.,)



e) Compression ratio = 131.03



f) Compression ratio = 201.39

The Original DCT-based JPEG Algorithm Applied to a Color Image

The JPEG2000 standard (cont.,)



d) A 128x128 subimage cropped from the standard JPEG image and enlarged to 256x256 using zero-order hold



e) A 128x128 subimage cropped from the JPEG2000 image and enlarged to 256x256 using zero order hold

Note: The JPEG2000 image is much smoother, even with the zero-order hold enlargement

References (videos)

https://www.youtube.com/watch?v=CPT4FSkFUgs&list=PLpsTn9TA_Q8VMDyOPrDKmSJYt1DLgDZU4&index=1

https://www.youtube.com/watch?v=Kv1Hiv3ox8I&t=54s

Review questions

1. Why RGB is converted to YCr Cb in JPEG compression standard?
2. What is the benefit of the DCT step in Jpeg compression standard?
3. What is the benefit of using Zigzag scan in Jpeg compression standard?
4. Explain how Jpeg compression standard works, and prove your answer by drawing.
5. Compare between Jpeg and Jpeg 2000 standards.

Questions

Thank you